

Zadanie 2 – Optymalizacja eliminacji Gaussa

1. Cel

Celem zadania była implementacja i optymalizacja algorytmu eliminacji Gaussa (bez pivotingu) pod kątem architektury posiadanego procesora, weryfikacja poprawności wyniku względem wersji referencyjnej oraz pomiar wydajności w GFLOP/s.

2. Procesor

```
zad2 on  main [!?] via C v16.1.1-gcc
> lscpu | grep -E "Model name|CPU max MHz|Core|Thread|L1d|L2|L3";
Model name:                Intel(R) Core(TM) i5-8300H CPU @ 2.30GHz
Thread(s) per core:        2
Core(s) per socket:        4
CPU max MHz:               4000,0000
L1d cache:                 128 KiB (4 instances)
L2 cache:                  1 MiB (4 instances)
L3 cache:                  8 MiB (1 instance)
```

Pomiary wykonano na procesorze **Intel Core i5-8300H** (Coffee Lake):

Parametr	Wartość
Rdzenie / wątki	4 / 8
Taktowanie	2.3 GHz (turbo do 4.0 GHz)
Jednostki wektorowe	SSE, AVX, AVX2 , FMA (rejestry 256-bit)
L1d	32 KB na rdzeń
L2	256 KB na rdzeń
L3	8 MB wspólne

Procesor obsługuje AVX2 z FMA, czyli na rejestrach 256-bitowych mieszczą się 4 liczby **double**, a instrukcja FMA wykonuje mnożenie i dodawanie w jednym kroku. Teoretyczny szczyt jednego rdzenia to 2 jednostki FMA × 4 double × 2 operacje = 16 FLOP/cykl, co przy 4.0 GHz daje ok. **64 GFLOP/s** (1 wątek, double precision).

3. Implementacja

Kod znajduje się w pliku **ge.c** i zawiera dwie wersje algorytmu:

Wersja referencyjna `ge_ref` – klasyczna potrójna pętla:

```
for (k = 0; k < SIZE; k++)
  for (i = k+1; i < SIZE; i++) {
    multiplier = A[i][k] / A[k][k];
    for (j = k+1; j < SIZE; j++)
      A[i][j] = A[i][j] - A[k][j] * multiplier;
  }
```

Wersja zoptymalizowana `ge_opt` – wzorowana na instruktażu how-to-optimize-gemm oraz laboratoriach. Zastosowane optymalizacje i miejsca w kodzie, gdzie się znajdują:

3.1 Wektoryzacja AVX2 + FMA – funkcje `UpdateKernel` i `UpdateKernelK`

Najbardziej wewnętrzna pętla algorytmu (tzw. kernel) jest napisana wprost na intrinsics AVX2. Jeden rejestr `__m256d` (256 bitów) mieści 4 liczby `double`, kernel używa dwóch rejestrów, więc przetwarza 8 elementów wiersza naraz. Operacja `c = c - p*m` (mnożenie + odejmowanie) jest wykonywana jedną instrukcją FMA `_mm256_fmadd_pd`:

```
/* wnetrze UpdateKernelK */
register __m256d c03 = _mm256_loadu_pd(c);      /* załaduj 4 double */
register __m256d c47 = _mm256_loadu_pd(c + 4); /* załaduj kolejne 4 */

for (r = 0; r < kdim; r++) {
  register __m256d mmult = _mm256_set1_pd(L[i * kdim + r]); /* broadcast
mnoznika */

  c03 = _mm256_fmadd_pd(u03[r], mmult, c03); /* c = c - u*m, 1
instrukcja FMA */
  c47 = _mm256_fmadd_pd(u47[r], mmult, c47);
}

_mm256_storeu_pd(c,      c03);
_mm256_storeu_pd(c + 4, c47);
```

Reszta wiersza niepodzielna przez 8 ("ogon") jest doliczana zwykłą pętlą skalarną, dzięki czemu kod działa dla dowolnego rozmiaru macierzy.

3.2 Aktualizacja blokowa rangi $KC=4$ – pętla główna w `ge_opt`

Zamiast aktualizować całą pozostałą macierz po każdym kroku eliminacji (aktualizacja rangi 1), główna pętla idzie po `k0` co $KC=4$ kroki: najpierw faktoryzowany jest wąski panel 4 kolumn, a potem macierz resztkowa jest aktualizowana **raz za wszystkie 4 kroki**: $A_{22} = A_{22} - L_{21} * U_{12}$ (czyli małe mnożenie macierzy, jak w GEMM). Każdy element `A22` jest dzięki temu ładowany z pamięci raz na 4 kroki zamiast co krok:

```
for (k0 = 0; k0 < SIZE; k0 += KC) {          /* glowna petla co KC krokow */
  ...
```

```

/* faktoryzacja malego panelu k0 ... kend-1 */
for (k = k0; k < kend; k++) { ... }

/* aktualizacja blokowa trailing matrix: A22 = A22 - L21 * U12 */
if (kend < SIZE) {
    PackLPanel(SIZE - kend, kb, A, SIZE, kend, k0, packed_l_all);
    for (jj = kend; jj < SIZE; jj += NC) {
        PackUPanel(kb, jb, A, SIZE, k0, jj, packed_u);
        for (ii = kend; ii < SIZE; ii += MC) {
            UpdateKernelK(ib, jb, kb, &A[IDX(ii, jj, SIZE)], SIZE, packed_u,
L);
        }
    }
}
}

```

3.3 Blokowanie pod cache – stałe MC, NC, KC i pętle po **jj**, **ii**

Aktualizacja jest dzielona na bloki dopasowane do hierarchii cache i5-8300H – widać to w pętlach **jj += NC** i **ii += MC** powyżej oraz w stałych na górze pliku:

```

/* parametry blokowania dobrane pod cache:
 * L1d = 32KB, L2 = 256KB, L3 = 8MB */
#define NR 8      /* szerokosc kernela: 8 doubli = 2 rejestry YMM */
#define MC 128    /* wysokosc bloku wierszy */
#define NC 256    /* szerokosc bloku kolumn */
#define KC 4      /* ranga aktualizacji blokowej */

```

Panel U ($KC \times NC \times 8 \text{ B} = 8 \text{ KB}$) mieści się w L1 (32 KB), blok roboczy $MC \times NC$ (256 KB) w L2/L3. Procesor "domiela" jeden blok do końca, zanim weźmie następny, więc dane nie wypadają z cache w trakcie obliczeń.

3.4 Packing – funkcje **PackUPanel**, **PackLPanel**, **PackPivotRow**, **PackMultipliers**

Dane potrzebne kernelowi są przed obliczeniami kopiowane do małych, **ciągłych** buforów. Np. mnożniki leżą w macierzy w kolumnie (co $SIZE \times 8$ bajtów, każdy w innej linii cache) – po spakowaniu leżą obok siebie i czytają się sekwencyjnie:

```

static void PackLPanel(int m, int kdim, const double *A, int SIZE,
                      int row0, int col0, double *L)
{
    for (i = 0; i < m; i++) {
        for (r = 0; r < kdim; r++) {
            double pivot = A[IDX(col0 + r, col0 + r, SIZE)];
            L[i * kdim + r] = A[IDX(row0 + i, col0 + r, SIZE)] / pivot; /*
mnoznik */
        }
    }
}

```

Przy okazji packingu liczone są od razu mnożniki (dzielenie przez pivot), więc kernel wykonuje już tylko operacje FMA.

3.5 Kompilacja

Kompilacja: `gcc -O2 -mavx2 -mfma ge.c -o ge` (flagi dopasowane do posiadanego procesora – AVX2 i FMA, bez AVX-512, którego i5-8300H nie ma). Wszystkie pomiary jednowątkowe.

4. Weryfikacja poprawności

Program dla zadanego rozmiaru wypełnia dwie identyczne macierze losowymi wartościami (`srand(1)`), wykonuje na jednej `ge_ref`, na drugiej `ge_opt`, a następnie porównuje wszystkie elementy i wypisuje maksymalny błąd względny. Wynik uznawany jest za poprawny, gdy błąd $< 1e-6$. Różnice rzędu $1e-11$ – $1e-7$ wynikają wyłącznie z innej kolejności operacji zmiennoprzecinkowych (blokowanie + FMA), nie z błędu algorytmu. **Weryfikacja przeszła dla wszystkich testowanych rozmiarów (200–2000).**

5. Szacowanie liczby operacji

Dla kroku k wykonuje się $(n-k-1)$ dzieleni (mnożniki) oraz $2 \cdot (n-k-1)^2$ mnożeń i odejmowań. Po zsumowaniu po wszystkich k :

$$\text{FLOP}(n) = n(n-1)(2n-1)/3 + n(n-1)/2 \approx (2/3) n^3$$

GFLOP/s liczone jest jako $\text{FLOP}(n) / \text{czas} / 1e9$.

6. Wyniki

```
zad2 on main [!?] via C v16.1.1-gcc
```

```
./run.sh
```

Eliminacja Gaussa - Intel Core i5-8300H, gcc -O2 -mavx2 -mfma, 1 watek

Szczyt teoretyczny 1 rdzenia: 64.0 GFLOP/s (4.0 GHz x 2 FMA x 4 double x 2 op)

SIZE	ref [GF/s]	opt [GF/s]	speedup	wysycenie	max blad	weryf
200	4.46	11.68	2.6x	18.2%	8.4e-10	OK
400	4.22	11.92	2.8x	18.6%	6.1e-10	OK
600	3.21	12.40	3.9x	19.4%	6.1e-11	OK
800	3.85	13.97	3.6x	21.8%	5.9e-09	OK
1000	3.37	8.87	2.6x	13.9%	1.1e-07	OK
1200	2.88	7.26	2.5x	11.4%	2.8e-09	OK
1400	2.72	5.76	2.1x	9.0%	1.6e-08	OK
1600	2.57	5.20	2.0x	8.1%	5.8e-08	OK
1800	2.46	4.99	2.0x	7.8%	1.2e-08	OK
2000	2.45	4.94	2.0x	7.7%	5.3e-08	OK

7. Wnioski

- **Wersja zoptymalizowana jest 2–3 razy szybsza od referencyjnej** i osiąga do ~14 GFLOP/s, czyli ok. 22% teoretycznego szczytu rdzenia (64 GFLOP/s).
- **Eliminacja Gaussa ma niską intensywność arytmetyczną** – aktualizacja rangi 1 wykonuje tylko 2 FLOP na każdy załadowany i zapisany element, dlatego nie da się osiągnąć wysycenia FLOPS-ów takiego jak w GEMM. Aktualizacja blokowa rangi KC częściowo to poprawia (2*KC FLOP na element).
- **Widać wyraźny wpływ hierarchii pamięci**: dla SIZE ≤ 1000 macierz (≤ 8 MB) mieści się w L3 i wydajność wynosi 11–13 GFLOP/s; dla większych rozmiarów macierz przestaje się mieścić w L3 i wydajność spada do ~5–6 GFLOP/s, bo algorytm staje się ograniczony przepustowością RAM.
- **Wektoryzacja AVX2 + FMA daje największy pojedynczy zysk** – kernel przetwarza 4 double na instrukcję, a FMA łączy mnożenie z odejmowaniem, co w porównaniu do wersji skalarnej zmniejsza liczbę instrukcji ~8 razy.
- Wyniki obu wersji są **identyczne z dokładnością do błędów zaokrągleń**, co potwierdza poprawność optymalizacji.